Լաբորատոր աշխատանք 6 — OOP (C++)

Աշխատանքի նպատակ

Ստեղծել համալսարանի գնահատականների վերահսկման համակարգ՝ կիրառելով դասեր, օբյեկտների զանգված (vector), մուտքային տվյալների ստուգում և դասի մեթոդներով իրականացված հիմնական տրամաբանություն։

Խնդրի պահանջ

Սահմանել Student դասը private դաշտերով՝ id, name, midterm, labPoints, exam, absences։ Ստեղծել 15 ուսանող։

Միավորների սահմանները պետք է ստուգվեն՝ միջանկյալ՝ առավելագույնը 20, լաբորատոր՝ 30, քննություն՝ 50։

Եթե բացակայությունները 6-ից շատ են, ուսանողը չի կարող մասնակցել քննությանը (քննության միավորը 0)։

Հաշվել վերջնական միավորը, տառային գնահատականը, գտնել չհաղթահարած ուսանողներին և հաշվարկել բաղադրիչների միջին միավորները։

Կազմել վարկանիշային ցուցակ՝ վերջնական միավորով, իսկ հավասարության դեպքում՝ լաբորատոր միավորով։

Առաջադրանք (C++ լուծում)

```
#include <iostream>
#include <vector>
#include <string>
#include <stdexcept>
#include <algorithm>
#include <iomanip>

class Student {
private:
    int id;
    std::string name;
    double midterm;   // max 20
    double labPoints; // max 30
    double exam;      // max 50
    int absences;

    static void validateRange(double value, double min, double max, const std::string& field) {
        if (value < min || value > max) {
            throw std::invalid_argument(field + " has invalid value");
        }
    }

public:
    Student(int id, const std::string& name, double midterm, double labPoints, double exam, int absenc
        : id(id), name(name), midterm(0), labPoints(0), exam(0), absences(0) {
        setMidterm(midterm);
        setLabPoints(labPoints);
        setAbsences(absences);
        setExam(exam);
    }

    void setMidterm(double value) { validateRange(value, 0, 20, "midterm"); midterm = value; }
    void setLabPoints(double value) { validateRange(value, 0, 30, "labPoints"); labPoints = value; }
    void setAbsences(int value) { if (value < 0) throw std::invalid_argument("absences < 0"); absences
    void setExam(double value) {
        if (absences > 6) { exam = 0; return; }
        validateRange(value, 0, 50, "exam");
```

```
        exam = value;
    }

    int getId() const { return id; }
    const std::string& getName() const { return name; }
    double getMidterm() const { return midterm; }
    double getLabPoints() const { return labPoints; }
    double getExam() const { return exam; }
    int getAbsences() const { return absences; }

    bool canTakeExam() const { return absences <= 6; }
    double finalScore() const { return midterm + labPoints + exam; }

    char letterGrade() const {
        double s = finalScore();
        if (s >= 90) return "A"[0];
        if (s >= 80) return "B"[0];
        if (s >= 70) return "C"[0];
        if (s >= 60) return "D"[0];
        return "F"[0];
    }

    bool isFailed() const { return finalScore() < 60; }
};

int main() {
    std::vector<Student> students;
    students.reserve(15);

    students.emplace_back(1, "Ani", 18, 27, 40, 2);
    students.emplace_back(2, "Aram", 12, 22, 30, 1);
    students.emplace_back(3, "Mariam", 20, 30, 48, 0);
    students.emplace_back(4, "Gor", 9, 18, 25, 3);
    students.emplace_back(5, "Narek", 15, 20, 33, 6);
    students.emplace_back(6, "Lilit", 17, 28, 45, 2);
    students.emplace_back(7, "Sona", 10, 19, 28, 4);
    students.emplace_back(8, "Karen", 14, 24, 35, 5);
    students.emplace_back(9, "Hasmik", 19, 26, 41, 1);
    students.emplace_back(10, "David", 11, 16, 27, 7); // exam forced to 0
    students.emplace_back(11, "Tigran", 13, 21, 31, 2);
    students.emplace_back(12, "Emma", 16, 25, 39, 0);
    students.emplace_back(13, "Arsen", 8, 14, 20, 3);
    students.emplace_back(14, "Mane", 18, 29, 46, 1);
    students.emplace_back(15, "Levon", 7, 12, 15, 8); // exam forced to 0

    double midAvg = 0, labAvg = 0, examAvg = 0;
    for (const auto& s : students) {
        midAvg += s.getMidterm();
        labAvg += s.getLabPoints();
        examAvg += s.getExam();
    }
    midAvg /= students.size();
    labAvg /= students.size();
    examAvg /= students.size();

    std::cout << "Failed students:\n";
    for (const auto& s : students) {
        if (s.isFailed()) {
            std::cout << s.getId() << " - " << s.getName() << " (" << s.finalScore() << ")\n";
        }
    }

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "\nComponent averages:\n";
    std::cout << "Midterm: " << midAvg << "\nLab: " << labAvg << "\nExam: " << examAvg << "\n";

    std::sort(students.begin(), students.end(), [](const Student& a, const Student& b) {
```

```
        if (a.finalScore() == b.finalScore()) return a.getLabPoints() > b.getLabPoints();
        return a.finalScore() > b.finalScore();
    });

    std::cout << "\nRanking list:\n";
    for (size_t i = 0; i < students.size(); ++i) {
        const auto& s = students[i];
        std::cout << (i + 1) << ". " << s.getName()
                  << " | score: " << s.finalScore()
                  << " | grade: " << s.letterGrade() << "\n";
    }

    return 0;
}
```